3. `tools.py` expliqué

Nutri-Arbitre — fiche de révision · étape 3 sur 5 du trajet des données (download → clean → **tools** → garde_fou → agent) · **briques 1 et 8**

> **En une phrase :** ce fichier contient les **outils** que Qwen peut appeler. Ce sont des fonctions Python qui font les calculs sur le tableau propre. **Ce sont elles, et elles seules, qui produisent les chiffres** : Qwen choisit quel outil appeler, mais il ne calcule jamais lui-même.
> **Brique 1** (votre propre outil) : ces fonctions, appelées par le modèle, avec une trace.
> **Brique 8** (des chiffres avec leur incertitude) : chaque moyenne vient avec sa marge d'erreur, et la comparaison bio / non bio dit si l'écart est net.

Vue d'ensemble : 4 outils, 6 aides, 1 « mode d'emploi »

Partie	Lignes	Rôle
Les **4 outils** : `decrire`, `comparer`, `classement_marques`, `compter`	63 à 153	Les calculs que Qwen peut demander.
Les **fonctions d'aide**, dont le nom commence par `_`	18 à 60	Vérifier les paramètres, filtrer, arrondir, calculer la marge. Le `_` est une convention : « fonction interne, pas un outil ».
`FONCTIONS` et `SCHEMAS`	156 à 185	Le « mode d'emploi » des outils, envoyé à Qwen pour qu'il sache lesquels existent et quels paramètres leur donner.

Les fonctions d'aide (lignes 18 à 60)

- **`_verifier_nutriment`** : si Qwen demande un nutriment qui n'existe pas (`"vitamine_c"`), on déclenche une erreur avec la liste des choix possibles. Cette erreur est renvoyée à Qwen (voir `agent.py`), qui peut alors corriger son appel.
- **`_booleen`** : un petit modèle envoie parfois `"false"` en texte au lieu de `false`. Or en Python, `bool("false")` vaut **True**, car tout texte non vide est « vrai ». Cette fonction lit le texte correctement, ce qui évite d'inverser le filtre bio.
- **`_filtrer`** : garde seulement les produits bio (ou non bio), ou d'une lettre de Nutri-Score. C'est la technique du **masque** vue dans `clean.py` : `df[df["bio"] == True]`. Si un filtre vaut `None`, on ne filtre pas.
- **`_arrondi`** : arrondit à 2 décimales, et renvoie `None` s'il n'y a pas de valeur. Les résultats sont donc lisibles et le cas « aucune valeur » ne plante pas.
- **`_erreur_type`** et **`_intervalle`** : la brique 8, expliquée ci-dessous.

La notion clé (brique 8) : la marge d'erreur

> **Le problème :** nos 953 céréales ne sont qu'un *échantillon* de toutes les céréales vendues. Avec un autre échantillon, la moyenne aurait été un peu différente. De combien ? C'est ce que dit la marge d'erreur, exactement comme dans un sondage (« 52 % ± 3 points »).

```
46 def _erreur_type(valeurs):
50     if len(valeurs) < 2:
51         return None
52     return valeurs.std() / math.sqrt(len(valeurs))
53
55 def _intervalle(centre, erreur_type):
59     marge = Z_95 * erreur_type
60     return marge, [centre - marge, centre + marge]
```

> 1. **L'écart-type** (`std()`) mesure si les valeurs sont dispersées ou regroupées autour de la moyenne.
> 2. **L'erreur type** = écart-type ÷ √n. Elle mesure la précision *de la moyenne*. Plus il y a de produits, plus elle est petite : la moyenne de 500 produits est plus fiable que celle de 5.
> 3. **La marge à 95 %** = 1,96 × erreur type. On peut dire : « la vraie moyenne a 95 % de chances d'être entre moyenne − marge et moyenne + marge ». Ce sont les bornes de l'**intervalle de confiance**.
>
> **Cas limite :** avec une seule valeur, on ne peut pas mesurer de dispersion. La fonction renvoie `None` au lieu d'inventer une marge.
>
> **D'où vient 1,96 ?** Avec beaucoup de produits, les moyennes possibles se répartissent en « courbe en cloche » (loi normale). 95 % de cette courbe se trouve à moins de 1,96 erreur type du centre.

Les 4 outils

`decrire` (lignes 63 à 79) : un nutriment en détail

Elle donne le nombre de produits `n`, la moyenne avec sa marge et son intervalle, la médiane, le minimum et le maximum. Les filtres `bio` et `nutriscore` sont facultatifs. `dropna()` retire les cases vides *de ce nutriment seulement*.

Exemple réel : `decrire("fibres", nutriscore="e")` → n = 29, moyenne 4,34 g ± 0,61 g, intervalle [3,73 ; 4,95].

`comparer` (lignes 82 à 125) : la moyenne par groupe

1. On garde les produits qui ont une valeur pour ce nutriment.

2. `groupby(groupe)` découpe le tableau en paquets : un paquet bio et un non bio, ou un par lettre de Nutri-Score. Pour chaque paquet, on calcule n, la moyenne et la marge.
3. **Pour le Nutri-Score**, on ajoute `sans_nutriscore` : le nombre de produits écartés parce qu'ils n'ont pas de lettre. Sans ça, la somme des groupes (928) ne correspondrait pas au total (953) sans explication.
4. **Pour le bio**, on calcule l'écart bio – non bio **et sa propre marge** :

```
118  marge, intervalle = _intervalle(ecart, math.sqrt(et_bio ** 2 + et_non_bio ** 2))
122  sortie["difference_nette"] = not (intervalle[0] <= 0 <= intervalle[1])
```

- L'erreur type d'un écart entre deux groupes indépendants vaut **$\sqrt{(\text{erreur type}_1^2 + \text{erreur type}_2^2)}$** : les deux incertitudes s'additionnent.
- **`difference_nette`** : si l'intervalle de l'écart contient 0, il est possible qu'il n'y ait en réalité *aucune* différence, et que l'écart observé vienne du hasard de l'échantillon. On ne peut alors pas conclure (`false`). Si 0 est en dehors, l'écart est net (`true`). S'il y a trop peu de produits pour calculer une marge, la réponse est `None` : on ne sait pas.

`classement_marques` (lignes 128 à 148) : les marques les plus (ou moins) riches

Quatre étapes, numérotées dans le code : garder les produits qui ont une marque et une valeur ; calculer par marque le nombre de produits (`count`) et la moyenne (`mean`) ; écarter les marques de moins de `min_produits` produits (5 par défaut, car une moyenne sur 1 produit ne dit rien) ; trier et garder les `n` premières.

Exemple réel : les 3 marques les moins sucrées sont jardin bio etic (6,5 g, 5 produits), markal (6,8 g, 7 produits) et celnat (7,4 g, 21 produits).

`compter` (lignes 151 à 153) : combien de produits

Elle filtre, puis compte les lignes avec `len`. *Exemple réel :* 543 produits bio, 270 notés A.

Le « mode d'emploi » pour Qwen (lignes 156 à 185)

- **`FONCTIONS`** est un dictionnaire « nom de l'outil → fonction Python ». Quand Qwen demande `"comparer"`, `agent.py` y trouve la fonction `comparer` à exécuter.
- **`SCHEMAS`** décrit chaque outil au format **JSON Schema** attendu par Ollama : son nom, une phrase qui explique ce qu'il fait, ses paramètres, et lesquels sont obligatoires (`required`). **C'est tout ce que Qwen sait des outils** : il ne voit jamais le code Python, seulement ces descriptions.
- Les **`enum`** donnent la liste fermée des valeurs permises (les 8 nutriments, les lettres a à e). Ça aide le modèle à ne pas inventer de nom de nutriment.
- `_schema` est une petite fonction qui évite de recopier quatre fois la même structure.

Ce qui s'est vraiment passé (1er novembre 2026), question « Les céréales bio contiennent-elles moins de sucre ? ». Qwen a appelé `comparer(nutriment="sucres", groupe="bio")`, qui a renvoyé :

Groupe	n	Moyenne	Marge 95 %
non bio	410	18,59 g	± 0,82 g
bio	543	13,17 g	± 0,69 g
écart		**–5,42 g**	± 1,07 g → [–6,49 ; –4,35] → net

Réponse de Qwen : « Oui, les céréales bio contiennent en moyenne moins de sucre que les autres (13,2 g contre 18,6 g pour 100 g). Cet écart significatif de plus de 5 g est statistiquement réel et ne vient pas du hasard. » La trace est enregistrée dans `traces/trace_20261101_125333.json`.

Limite : Qwen n'a pas écrit la marge, alors que le prompt le demande. Un petit modèle ne suit pas toujours toutes les consignes.

Ce que les données ne permettent pas de conclure (brique 8) : la marge ne mesure que le hasard de l'échantillon, pas son biais. Nos 1 000 produits sont ceux qu'Open Food Facts a renvoyés en premier (57 % de bio), et pas un tirage au hasard parmi toutes les céréales vendues en France. L'écart est net *dans notre échantillon*, mais on ne peut pas l'affirmer pour tout le marché français. Et on ne peut pas dire que le bio *cause* moins de sucre : les céréales bio sont peut-être surtout des mueslis et des flocons, moins sucrés par nature.

Questions d'examinateur

1. Pourquoi Qwen ne calcule-t-il pas les moyennes lui-même ?
2. Que se passe-t-il si Qwen demande `decrire("vitamine_c")` ?
3. Pourquoi a-t-on besoin de `_booleen` ? Que vaut `bool("false")` en Python ?
4. Comment calcule-t-on la marge d'erreur à 95 % ? Pourquoi diminue-t-elle quand il y a plus de produits ?
5. Que veut dire `difference_nette: false` ?
6. Pourquoi la marge vaut-elle `None` quand il n'y a qu'une seule valeur ?
7. Pourquoi `classement_marques` ignore-t-il les marques de moins de 5 produits ?
8. Comment Qwen sait-il quels outils existent ?
9. L'écart de sucre entre bio et non bio est « net ». Peut-on en conclure que, en France, les céréales bio sont moins sucrées ?

Essaie d'y répondre avec tes mots avant de lire le corrigé à la page suivante.

Corrigé

1. **Qwen ne calcule pas** : un modèle de langage prédit du texte, il ne fait pas de vrai calcul. Il peut écrire une moyenne fausse avec assurance. C'est le principe du projet : le modèle choisit l'outil, le code calcule, et le garde-fou vérifie que les chiffres de la réponse viennent bien des outils.
2. `"vitamine_c"` : `_verifier_nutriment` déclenche une `ValueError` avec la liste des nutriments possibles. `agent.py` l'attrape et la renvoie à Qwen sous la forme `{"erreur": ...}` : le programme ne plante pas, et Qwen peut corriger son appel. C'est vérifié par un test.
3. `_booleen` : `bool("false")` vaut `True`, parce que tout texte non vide est « vrai » en Python. Si Qwen envoie `"false"` en texte, le filtre « non bio » deviendrait « bio » : la réponse serait l'inverse de la vérité. `_booleen` lit le texte.
4. **La marge** : 1,96 × écart-type ÷ √n. Le √n est au dénominateur : quand n augmente, la marge diminue. Avec plus de produits, le hasard de l'échantillon pèse moins, et la moyenne est plus précise.
5. `difference_nette: false` : l'intervalle de l'écart contient 0. Il est donc possible que la vraie différence soit nulle, et que l'écart observé ne vienne que du hasard. On ne peut pas conclure.
6. **Une seule valeur** : la marge repose sur la dispersion des valeurs. Avec une seule valeur, il n'y a pas de dispersion à mesurer. Plutôt que d'inventer un chiffre, l'outil renvoie `None` : « on ne sait pas ».
7. **Moins de 5 produits** : une moyenne sur 1 ou 2 produits ne représente pas une marque. Une marque avec un seul muesli sans sucre finirait première du classement par hasard.
8. **Comment Qwen connaît les outils** : par `SCHEMAS`, envoyé à chaque appel. Il y lit le nom de chaque outil, ce qu'il fait et ses paramètres. Il ne voit jamais le code Python.
9. **Peut-on généraliser à la France ?** Non. La marge ne mesure que le hasard de l'échantillon, pas son biais : nos produits ne sont pas tirés au hasard (57 % de bio, ceux qu'Open Food Facts renvoie en premier). L'écart est net dans notre échantillon, mais on ne peut pas l'affirmer pour tout le marché. Et ce n'est pas une preuve de cause : les céréales bio sont peut-être surtout d'un type moins sucré (mueslis, flocons). C'est la « phrase sur ce que les données ne permettent pas de conclure » demandée par la brique 8.